

Slip 1

Q.1

Question:

Write a program to calculate perimeter and area of rectangle.

(Hint: area = length * breadth, perimeter = 2 * (length + breadth))

CODE:

```
import java.util.Scanner;
class Rect {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double length, breadth;
        if (args.length >= 2) {
            length = Double.parseDouble(args[0]);
            breadth = Double.parseDouble(args[1]);
        } else {
            System.out.print("Enter length: ");
            length = sc.nextDouble();
            System.out.print("Enter breadth: ");
            breadth = sc.nextDouble();
        }
        double area = length * breadth;
        double perimeter = 2 * (length + breadth);
        System.out.println("Area = " + area);
        System.out.println("Perimeter = " + perimeter);
    }
}
```

OUTPUT:

```
Enter length: 5
Enter breadth: 3
Area = 15.0
Perimeter = 16.0
```

Q.2

Question:

Create an abstract class “Order” having members id, description.

Create two subclasses “PurchaseOrder” and “SalesOrder” having members customer name and vendor name respectively.

Define methods accept and display in all cases.

Create 3 objects each of PurchaseOrder and SalesOrder and accept and display details.

Program:

```
import java.util.Scanner;
abstract class Order {
    int id;
    String description;
    Order(int id, String description) {
```

```

        this.id = id;
        this.description = description;
    }
    abstract void display();
}

class PurchaseOrder extends Order {
    String customer;
    PurchaseOrder(int id, String description, String customer) {
        super(id, description);
        this.customer = customer;
    }
    void display() {
        System.out.println("Purchase Order -> ID: " + id + ", Description: " + description + ",
Customer: " + customer);
    }
}

class SalesOrder extends Order {
    String vendor;
    SalesOrder(int id, String description, String vendor) {
        super(id, description);
        this.vendor = vendor;
    }
    void display() {
        System.out.println("Sales Order -> ID: " + id + ", Description: " + description + ", Vendor: " +
vendor);
    }
}

class TestOrders {
    public static void main(String[] args) {
        PurchaseOrder[] po = {
            new PurchaseOrder(1, "Stationery", "Alice"),
            new PurchaseOrder(2, "Chairs", "Bob"),
            new PurchaseOrder(3, "Books", "Carol")
        };
        SalesOrder[] so = {
            new SalesOrder(11, "Laptop", "VendorA"),
            new SalesOrder(12, "Mouse", "VendorB"),
            new SalesOrder(13, "Keyboard", "VendorC")
        };
        for (PurchaseOrder p : po) {
            p.display();
        }
        for (SalesOrder s : so) {
            s.display();
        }
    }
}

```

OUTPUT:

Purchase Order -> ID: 1, Description: Stationery, Customer: Alice
Purchase Order -> ID: 2, Description: Chairs, Customer: Bob
Purchase Order -> ID: 3, Description: Books, Customer: Carol
Sales Order -> ID: 11, Description: Laptop, Vendor: VendorA
Sales Order -> ID: 12, Description: Mouse, Vendor: VendorB
Sales Order -> ID: 13, Description: Keyboard, Vendor: VendorC

Slip 2

Q.1

Question:

Write a menu driven program to perform the following operations:

- i. Calculate the volume of cylinder (Volume = $\pi \times r^2 \times h$)
- ii. Find the factorial of a number
- iii. Check whether a number is Armstrong or not
- iv. Exit

Program:

```
import java.util.Scanner;
class Slip2Q1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        while (true) {
            System.out.println("1. Volume of Cylinder");
            System.out.println("2. Factorial");
            System.out.println("3. Armstrong Check");
            System.out.println("4. Exit");
            System.out.print("Enter choice: ");
            int ch = sc.nextInt();
            if (ch == 1) {
                System.out.print("Enter radius and height: ");
                double r = sc.nextDouble(), h = sc.nextDouble();
                double volume = Math.PI * r * r * h;
                System.out.println("Volume = " + volume);
            } else if (ch == 2) {
                System.out.print("Enter number: ");
                int n = sc.nextInt();
                long fact = 1;
                for (int i = 2; i <= n; i++) fact *= i;
                System.out.println("Factorial = " + fact);
            } else if (ch == 3) {
                System.out.print("Enter number: ");
                int n = sc.nextInt(), temp = n, sum = 0;
                while (n > 0) {
                    int d = n % 10;
                    sum += d * d * d;
                    n /= 10;
                }
                System.out.println(temp + (sum == temp ? " is Armstrong" : " is not Armstrong"));
            } else break;
        }
    }
}
```

OUTPUT

1. Volume of Cylinder

2. Factorial

3. Armstrong Check

4. Exit

Enter choice: 1

Enter radius and height: 3 5

Volume = 141.3716694115407

Enter choice: 2

Enter number: 5

Factorial = 120

Enter choice: 3

Enter number: 153

153 is Armstrong

Enter choice: 4

Q.2

Question:

Using marker interface, create a class Product (product_id, product_name, product_cost, product_quantity).

Define default and parameterized constructors.

Create objects of class Product and display the contents of each object.

Also display object count.

Program:

```
interface Marker {}

class Product implements Marker {
    static int count = 0;
    int id, qty;
    String name;
    double cost;
    Product() { this(0, "", 0, 0); }
    Product(int id, String name, double cost, int qty) {
        this.id = id;
        this.name = name;
        this.cost = cost;
        this.qty = qty;
        count++;
    }
    public String toString() {
        return "ID: " + id + ", Name: " + name + ", Cost: " + cost + ", Qty: " + qty;
    }
    public static void main(String[] args) {
        Product p1 = new Product(1, "Pen", 5.0, 10);
        Product p2 = new Product(2, "Book", 50.0, 5);
        Product p3 = new Product(3, "Pencil", 2.0, 20);
        System.out.println(p1);
        System.out.println(p2);
        System.out.println(p3);
        System.out.println("Object Count = " + Product.count);
    }
}
```

OUTPUT:

```
ID: 1, Name: Pen, Cost: 5.0, Qty: 10
ID: 2, Name: Book, Cost: 50.0, Qty: 5
ID: 3, Name: Pencil, Cost: 2.0, Qty: 20
Object Count = 3
```

Slip 3

Q.1

Question:

Write a program to accept the array elements and display them in reverse order.

Program:

```
import java.util.Scanner;

class ReverseArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter elements:");
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
        System.out.println("Array in reverse order:");
        for (int i = n - 1; i >= 0; i--) System.out.print(arr[i] + " ");
    }
}
```

Sample Output:

Enter number of elements: 5

Enter elements:

1 2 3 4 5

Array in reverse order:

5 4 3 2 1

Q.2

Question:

Write a Java program to create a Package “SY” which has a class SYMarks (members – ComputerTotal, MathsTotal, ElectronicsTotal).

Create another package TY which has a class TYMarks (members – Theory, Practicals).

Create n objects of Student class (having rollNumber, name, SYMarks and TYMarks).

Add the marks of SY and TY computer subjects and calculate the Grade (‘A’ for >= 70, ‘B’ for >= 60, ‘C’ for >= 50, Pass Class for >= 40 else ‘FAIL’) and display result.

Program:

```
sy/SYMarks.java
package sy;
public class SYMarks {
    public int computerTotal, mathsTotal, electronicsTotal;
    public SYMarks(int c, int m, int e) {
        computerTotal = c;
        mathsTotal = m;
        electronicsTotal = e;
    }
}

ty/TYMarks.java
package ty;
```

```

public class TYMarks {
    public int theory, practicals;
    public TYMarks(int th, int pr) {
        theory = th;
        practicals = pr; }
}

StudentMain.java
import sy.SYMarks;
import ty.TYMarks;
import java.util.Scanner;

class Student {
    int roll;
    String name;
    SYMarks sy;
    TYMarks ty;
    Student(int r, String n, SYMarks s, TYMarks t) {
        roll = r;
        name = n;
        sy = s;
        ty = t; }
}

class StudentMain {
    public static char grade(int total) {
        if (total >= 70) return 'A';
        if (total >= 60) return 'B';
        if (total >= 50) return 'C';
        if (total >= 40) return 'P';
        return 'F'; }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of students: ");
        int n = sc.nextInt();
        Student[] st = new Student[n];
        for (int i = 0; i < n; i++) {
            System.out.print("Enter roll and name: ");
            int r = sc.nextInt();
            String name = sc.next();
            System.out.print("Enter SY computer, maths, electronics: ");
            int sc1 = sc.nextInt(), sm = sc.nextInt(), se = sc.nextInt();
            System.out.print("Enter TY theory and practicals: ");
            int tt = sc.nextInt(), tp = sc.nextInt();
            st[i] = new Student(r, name, new SYMarks(sc1, sm, se), new TYMarks(tt, tp));
        }
        for (Student s : st) {
            int total = s.sy.computerTotal + s.ty.theory;
            System.out.println(s.roll + " " + s.name + " Total=" + total + " Grade=" + grade(total));
        }
    }
}

```

Sample Output:

Enter number of students: 2

Enter roll and name: 1 Alice

Enter SY computer, maths, electronics: 80 70 75

Enter TY theory and practicals: 65 60

Enter roll and name: 2 Bob

Enter SY computer, maths, electronics: 50 60 55

Enter TY theory and practicals: 40 45

1 Alice Total=145 Grade=A

2 Bob Total=90 Grade=A

Slip 4

Q.1

Question:

Create an employee class (id, name, deptname, salary).

Use this keyword to initialize instance variables.

Keep a count of objects created using static variable and display object count after each object is created.

Program:

```
class Employee {
    int id;
    String name, dept;
    double salary;
    static int count = 0;

    Employee() { this(0, "", "", 0); }
    Employee(int id, String name, String dept, double salary) {
        this.id = id;
        this.name = name;
        this.dept = dept;
        this.salary = salary;
        count++;
    }

    public String toString() {
        return "ID: " + id + ", Name: " + name + ", Dept: " + dept + ", Salary: " + salary;
    }

    public static void main(String[] args) {
        Employee e1 = new Employee(1, "A", "CS", 50000);
        System.out.println(e1);
        System.out.println("Count = " + Employee.count);
        Employee e2 = new Employee(2, "B", "IT", 45000);
        System.out.println(e2);
        System.out.println("Count = " + Employee.count);
        Employee e3 = new Employee(3, "C", "ME", 40000);
        System.out.println(e3);
        System.out.println("Count = " + Employee.count);
    }
}
```

Sample Output:

ID: 1, Name: A, Dept: CS, Salary: 50000.0

Count = 1

ID: 2, Name: B, Dept: IT, Salary: 45000.0

Count = 2

ID: 3, Name: C, Dept: ME, Salary: 40000.0

Count = 3

Q.2

Question:

Write a program to read book information (bookid, bookname, bookprice, bookqty) in file "book.dat".

Write a menu driven program to perform the following operations using RandomAccessFile:

- i. Search for a specific book by name
- ii. Display all books and total cost

Program:

```
import java.io.RandomAccessFile;
import java.util.Scanner;

class BookDB {
    static final String FILE = "book.dat";

    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        RandomAccessFile f = new RandomAccessFile(FILE, "rw");
        while (true) {
            System.out.println("1. Add Book");
            System.out.println("2. Search by Name");
            System.out.println("3. Display All");
            System.out.println("4. Exit");
            System.out.print("Enter choice: ");
            int ch = sc.nextInt();
            sc.nextLine();
            if (ch == 1) {
                System.out.print("Enter id name price qty: ");
                int id = sc.nextInt();
                String name = sc.next();
                double price = sc.nextDouble();
                int qty = sc.nextInt();
                f.seek(f.length());
                f.writeInt(id);
                f.writeUTF(name);
                f.writeDouble(price);
                f.writeInt(qty);
            } else if (ch == 2) {
                System.out.print("Enter name to search: ");
                String key = sc.next();
                f.seek(0);
                boolean found = false;
                while (f.getFilePointer() < f.length()) {
                    int id = f.readInt();
                    String name = f.readUTF();
                    double price = f.readDouble();
                    int qty = f.readInt();
                    if (name.equalsIgnoreCase(key)) {
```



```

        System.out.println(id + " " + name + " " + price + " " + qty);
        found = true;
    }
}
if (!found) System.out.println("Not found");
} else if (ch == 3) {
    f.seek(0);
    double total = 0;
    while (f.getFilePointer() < f.length()) {
        int id = f.readInt();
        String name = f.readUTF();
        double price = f.readDouble();
        int qty = f.readInt();
        System.out.println(id + " " + name + " " + price + " " + qty);
        total += price * qty;
    }
    System.out.println("Total Cost = " + total);
} else break;
}
f.close();
}
}

```

Sample Output:

```

1. Add Book
2. Search by Name
3. Display All
4. Exit
Enter choice: 1
Enter id name price qty: 101 JavaBook 250.0 2
Enter choice: 1
Enter id name price qty: 102 DataStruct 300.0 1
Enter choice: 3
101 JavaBook 250.0 2
102 DataStruct 300.0 1
Total Cost = 800.0
Enter choice: 2
Enter name to search: JavaBook
101 JavaBook 250.0 2
Enter choice: 4

```

Slip 5

Q.1

Question:

Define Student class (roll_no, name, percentage) to create n objects of the Student class.

Accept details from the user for each object.

Define a static method "sortStudent" which sorts the array on the basis of percentage.

Program:

```
import java.util.Scanner;
```

```
import java.util.Arrays;
```

```
class Student {  
    int roll;  
    String name;  
    double percentage;  
  
    Student(int roll, String name, double percentage) {  
        this.roll = roll;  
        this.name = name;  
        this.percentage = percentage;  
    }  
  
    public String toString() {  
        return roll + " " + name + " " + percentage;  
    }  
  
    static void sortStudent(Student[] s) {  
        Arrays.sort(s, (a, b) -> Double.compare(b.percentage, a.percentage));  
    }  
  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter number of students: ");  
        int n = sc.nextInt();  
        Student[] st = new Student[n];  
        for (int i = 0; i < n; i++) {  
            System.out.print("Enter roll, name, percentage: ");  
            int r = sc.nextInt();  
            String name = sc.next();  
            double p = sc.nextDouble();  
            st[i] = new Student(r, name, p);  
        }  
        sortStudent(st);  
        System.out.println("Sorted Students:");  
        for (Student s : st) System.out.println(s);  
    }  
}
```

Sample Output:

Enter number of students: 3

Enter roll, name, percentage: 1 Alice 85

Enter roll, name, percentage: 2 Bob 78

Enter roll, name, percentage: 3 Carol 90

Sorted Students:

3 Carol 90.0

1 Alice 85.0

2 Bob 78.0

Q.2

Question:

Define a class MyNumber having one private int data member.

Write a default constructor to initialize it to 0 and another constructor to initialize it to a value (use this).

Write methods isNegative, isPositive, isZero, isOdd, isEven.

Create an object in main using command line argument to pass a value and perform the tests.

Program:

```
class MyNumber {  
    private int num;  
  
    MyNumber() { this(0); }  
    MyNumber(int num) { this.num = num; }  
  
    boolean isNegative() { return num < 0; }  
    boolean isPositive() { return num > 0; }  
    boolean isZero() { return num == 0; }  
    boolean isOdd() { return num % 2 != 0; }  
    boolean isEven() { return num % 2 == 0; }  
  
    public static void main(String[] args) {  
        int val = args.length > 0 ? Integer.parseInt(args[0]) : 0;  
        MyNumber n = new MyNumber(val);  
        System.out.println("Number: " + val);  
        System.out.println("Negative: " + n.isNegative());  
        System.out.println("Positive: " + n.isPositive());  
        System.out.println("Zero: " + n.isZero());  
        System.out.println("Odd: " + n.isOdd());  
        System.out.println("Even: " + n.isEven());  
    }  
}
```

Sample Output (for argument 5):

```
Number: 5  
Negative: false  
Positive: true  
Zero: false  
Odd: true  
Even: false
```

Slip 6

Q.1

Question:

Write a Java program to accept 5 numbers using command line arguments, sort and display them.

Program:

```
import java.util.Arrays;
```

```

class SortArgs {
    public static void main(String[] args) {
        int[] arr = new int[args.length];
        for (int i = 0; i < args.length; i++) arr[i] = Integer.parseInt(args[i]);
        Arrays.sort(arr);
        System.out.println("Sorted Numbers:");
        for (int n : arr) System.out.print(n + " ");
    }
}

```

Sample Output (for arguments 9 3 1 5 2):

Sorted Numbers:

1 2 3 5 9

Q.2

Question:

Write a Java program to display the system date and time in various formats using Date and SimpleDateFormat.

Program:

```

import java.util.Date;
import java.text.SimpleDateFormat;

class DateFormats {
    public static void main(String[] args) {
        Date d = new Date();
        System.out.println("Current date is : " + new SimpleDateFormat("dd/MM/yyyy").format(d));
        System.out.println("Current date is : " + new SimpleDateFormat("MM-dd-yyyy").format(d));
        System.out.println("Current date is : " + new SimpleDateFormat("EEEE MMMM dd
yyyy").format(d));
        System.out.println("Current date and time is : " + new SimpleDateFormat("E MMMM dd
HH:mm:ss z yyyy").format(d));
        System.out.println("Current date and time is : " + new SimpleDateFormat("dd/MM/yy
HH:mm:ss a Z").format(d));
        System.out.println("Current time is : " + new SimpleDateFormat("HH:mm:ss").format(d));
        System.out.println("Current week of year is : " + new SimpleDateFormat("w").format(d));
        System.out.println("Current week of month : " + new SimpleDateFormat("W").format(d));
        System.out.println("Current day of the year is : " + new SimpleDateFormat("D").format(d));
    }
}

```

Sample Output (will vary):

Current date is : 17/09/2025

Current date is : 09-17-2025

Current date is : Wednesday September 17 2025

Current date and time is : Wed September 17 14:35:20 IST 2025

Current date and time is : 17/09/25 14:35:20 PM +0530

Current time is : 14:35:20

Current week of year is : 38

Current week of month : 3

Slip 7

Q.1

Question:

Write a Java program that takes input as a person name in the format first, middle, last and print in the form last, first, middle (capitalize first letter of middle name).

Program:

```
import java.util.Scanner;

class NameFormat {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first, middle, last name: ");
        String first = sc.next(), middle = sc.next(), last = sc.next();
        middle = middle.substring(0,1).toUpperCase() + middle.substring(1).toLowerCase();
        System.out.println("Formatted: " + last + ", " + first + " " + middle);
    }
}
```

Sample Output:

```
Enter first, middle, last name: john michael doe
Formatted: doe, john Michael
```

Q.2

Question:

Define class EmailId with members username and password.

Accept values from the command line.

Throw user defined exceptions – InvalidUsernameException or InvalidPasswordException if username or password are invalid.

Program:

```
class InvalidUsernameException extends Exception {
    InvalidUsernameException(String msg) { super(msg); }
}

class InvalidPasswordException extends Exception {
    InvalidPasswordException(String msg) { super(msg); }
}

class EmailId {
    String username, password;
    EmailId(String u, String p) { username = u; password = p; }
    void validate() throws InvalidUsernameException, InvalidPasswordException {
        if (!username.contains("@")) throw new InvalidUsernameException("Username must contain @");
        if (password.length() < 6) throw new InvalidPasswordException("Password must be at least 6 chars");
    }
    public static void main(String[] args) {
```

```

try {
    if (args.length < 2) {
        System.out.println("Pass username and password as arguments.");
        return;
    }
    EmailId e = new EmailId(args[0], args[1]);
    e.validate();
    System.out.println("Login successful");
} catch (Exception ex) {
    System.out.println("Error: " + ex.getMessage());
}
}
}

```

Sample Output (arguments: user@mail pass123):

Login successful

Sample Output (arguments: user pass):

Error: Username must contain @

Slip 8

Q.1

Question:

Write a Java program that takes input as a person name in the format first, middle, last and print in the form last, first, middle (capitalize first letter of middle name).

Program:

(Same as Slip 7 Q.1) var bug slip NO 7 madhi Q1

Q.2

Question:

Define a class MyDate (day, month, year) with methods to accept and display a MyDate object. Throw user defined exception "InvalidDateException" if the date is invalid.

Program:

```
import java.util.Scanner;
```

```

class InvalidDateException extends Exception {
    InvalidDateException(String msg) { super(msg); }
}

```

```

class MyDate {
    int d, m, y;
    MyDate(int d, int m, int y) { this.d = d; this.m = m; this.y = y; }

    void validate() throws InvalidDateException {
        if (m < 1 || m > 12) throw new InvalidDateException("Invalid Month");
        if (d < 1 || d > 31) throw new InvalidDateException("Invalid Day");
        if ((m == 4 || m == 6 || m == 9 || m == 11) && d > 30)
            throw new InvalidDateException("Invalid Day for Month");
        if (m == 2) {

```

```

        boolean leap = (y % 4 == 0 && (y % 100 != 0 || y % 400 == 0));
        if ((leap && d > 29) || (!leap && d > 28))
            throw new InvalidDateException("Invalid Day for February");
    }
}

```

```

void display() { System.out.println("Date: " + d + "/" + m + "/" + y); }

```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter dd mm yyyy: ");
    int d = sc.nextInt(), m = sc.nextInt(), y = sc.nextInt();
    try {
        MyDate date = new MyDate(d, m, y);
        date.validate();
        date.display();
    } catch (InvalidDateException e) {
        System.out.println("Error: " + e.getMessage());
    }
}
}

```

Sample Output (valid):

Enter dd mm yyyy: 28 2 2024

Date: 28/2/2024

Sample Output (invalid):

Enter dd mm yyyy: 29 2 2021

Error: Invalid Day for February

Slip 9

Q.1

Question:

Define a class CricketPlayer (name, no_of_innings, no_of_times_notout, totalruns, bat_avg).

Create an array of n player objects.

Calculate the batting average for each player using static method avg().

Sort the array on the basis of average and display details.

Program:

```
import java.util.Scanner;
```

```
import java.util.Arrays;
```

```
class CricketPlayer {
```

```
    String name;
```

```
    int innings, notOut, runs;
```

```
    double avg;
```

```
    CricketPlayer(String n, int in, int no, int r) {
```

```
        name = n;
```

```
        innings = in;
```

```

        notOut = no;
        runs = r;
        avg = avg(runs, innings - notOut);
    }

    static double avg(int runs, int outs) { return outs == 0 ? runs : (double) runs / outs; }

    public String toString() {
        return name + " " + innings + " " + notOut + " " + runs + " Avg=" + avg;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of players: ");
        int n = sc.nextInt();
        CricketPlayer[] p = new CricketPlayer[n];
        for (int i = 0; i < n; i++) {
            System.out.print("Enter name innings notOut runs: ");
            p[i] = new CricketPlayer(sc.next(), sc.nextInt(), sc.nextInt(), sc.nextInt());
        }
        Arrays.sort(p, (a, b) -> Double.compare(b.avg, a.avg));
        System.out.println("Sorted Players by Average:");
        for (CricketPlayer c : p) System.out.println(c);
    }
}

```

Sample Output:

```

Enter number of players: 2
Enter name innings notOut runs: Virat 10 2 800
Enter name innings notOut runs: Rohit 10 1 700
Sorted Players by Average:
Virat 10 2 800 Avg=100.0
Rohit 10 1 700 Avg=77.77777777777777

```

Q.2

Question:

Create a GUI screen using layout managers to accept name, class, hobbies and display selected options in a textbox.

(GUI Output will depend on runtime, no console output)

Program:

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class UserForm extends JFrame implements ActionListener {
    JTextField nameField, classField;
    JCheckBox hobby1, hobby2, hobby3;
    JTextArea output;
}

```


JButton submit;

```
UserForm() {
    setLayout(new GridLayout(5, 2));
    add(new JLabel("Name:"));
    nameField = new JTextField(); add(nameField);
    add(new JLabel("Class:"));
    classField = new JTextField(); add(classField);
    hobby1 = new JCheckBox("Reading");
    hobby2 = new JCheckBox("Sports");
    hobby3 = new JCheckBox("Music");
    add(hobby1); add(hobby2); add(hobby3);
    submit = new JButton("Show");
    submit.addActionListener(this);
    add(submit);
    output = new JTextArea();
    add(output);
    setSize(300,300);
    setVisible(true);
    setDefaultCloseOperation(EXIT_ON_CLOSE);
}

public void actionPerformed(ActionEvent e) {
    String hobbies = "";
    if (hobby1.isSelected()) hobbies += "Reading ";
    if (hobby2.isSelected()) hobbies += "Sports ";
    if (hobby3.isSelected()) hobbies += "Music ";
    output.setText("Name: " + nameField.getText() + "\nClass: " + classField.getText() +
"\nHobbies: " + hobbies);
}

public static void main(String[] args) {
    new UserForm();
}
}
```

Slip 10

Q.1

Question:

Write a program for multilevel inheritance such that country is inherited from continent, state is inherited from country. Display place, state, country, continent.

Program:

```
class Continent {
    String cont;
    Continent(String c) { cont = c; }
}

class Country extends Continent {
```

```

String country;
Country(String c, String country) { super(c); this.country = country; }
}
class State extends Country {
String state;
State(String c, String country, String s) { super(c, country); state = s; }
void display(String place) {
    System.out.println("Place: " + place);
    System.out.println("State: " + state);
    System.out.println("Country: " + country);
    System.out.println("Continent: " + cont);
}
}
class MainClass {
    public static void main(String[] args) {
        State s = new State("Asia", "India", "Maharashtra");
        s.display("Pune");
    }
}

```

Sample Output:

```

Place: Pune
State: Maharashtra
Country: India
Continent: Asia

```

Q.2

Question:

Write a Java program to design a screen using AWT that takes username and password. If they are not same, raise an Exception. User has 3 login chances. Include clear button.

Program:

```

import java.awt.*;
import java.awt.event.*;

class LoginFrame extends Frame implements ActionListener {
    TextField userField, passField;
    Button login, clear;
    int attempts = 0;

    LoginFrame() {
        setLayout(new GridLayout(3, 2));
        add(new Label("Username:"));
        userField = new TextField(); add(userField);
        add(new Label("Password:"));
        passField = new TextField(); passField.setEchoChar('*'); add(passField);
        login = new Button("Login"); clear = new Button("Clear");
        add(login); add(clear);
        login.addActionListener(this);
    }
}

```

```
clear.addActionListener(e -> { userField.setText(""); passField.setText(""); });
setSize(300,150);
setVisible(true);
}
```

```
public void actionPerformed(ActionEvent e) {
    try {
        if (!userField.getText().equals(passField.getText())) {
            attempts++;
            throw new Exception("Invalid Credentials");
        }
        System.out.println("Login Successful");
        dispose();
    } catch (Exception ex) {
        System.out.println(ex.getMessage());
        if (attempts >= 3) {
            System.out.println("3 Attempts Over. Exiting...");
            dispose();
        }
    }
}
```

```
public static void main(String[] args) {
    new LoginFrame();
}
}
```

(Output is GUI-based; error messages printed in console)

Great  — here's **Slip 11 to Slip 15** in the same clean format (Question → Program → Output).
I've kept code well-formatted and sample outputs included wherever possible.

Slip 11

Q.1

Question:

Define an abstract class Staff with protected members id and name.

Define a parameterized constructor.

Define one subclass OfficeStaff with member department.

Create n objects of OfficeStaff and display all details.

Program:

```
import java.util.Scanner;

abstract class Staff {
    protected int id;
    protected String name;
    Staff(int id, String name) {
        this.id = id;
        this.name = name;
    }
    abstract void display();
}

class OfficeStaff extends Staff {
    String dept;
    OfficeStaff(int id, String name, String dept) {
        super(id, name);
        this.dept = dept;
    }
    void display() {
        System.out.println("ID: " + id + ", Name: " + name + ", Dept: " + dept);
    }
}

class StaffMain {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of staff: ");
        int n = sc.nextInt();
        OfficeStaff[] s = new OfficeStaff[n];
        for (int i = 0; i < n; i++) {
            System.out.print("Enter id, name, dept: ");
            int id = sc.nextInt();
            String name = sc.next();
            String dept = sc.next();
            s[i] = new OfficeStaff(id, name, dept);
        }
    }
}
```

```

        System.out.println("Staff Details:");
        for (OfficeStaff st : s) st.display();
    }
}

```

Sample Output:

```

Enter number of staff: 2
Enter id, name, dept: 1 Alice HR
Enter id, name, dept: 2 Bob IT
Staff Details:
ID: 1, Name: Alice, Dept: HR
ID: 2, Name: Bob, Dept: IT

```

Q.2

Question:

Define a class MyNumber having one private int data member.
 Write a default constructor to initialize it to 0 and another constructor to initialize it to a value.
 Write methods isNegative, isPositive, isZero, isOdd, isEven.
 Create an object in main using command line argument and perform the tests.
 Generate javadoc comments for all constructors and methods.

Program:

```

/**
 * MyNumber class performs checks on an integer value.
 */
class MyNumber {
    private int num;

    /** Default constructor initializes number to 0 */
    MyNumber() { this(0); }

    /** Parameterized constructor initializes number to given value */
    MyNumber(int num) { this.num = num; }

    /** @return true if number is negative */
    boolean isNegative() { return num < 0; }

    /** @return true if number is positive */
    boolean isPositive() { return num > 0; }

    /** @return true if number is zero */
    boolean isZero() { return num == 0; }

    /** @return true if number is odd */
    boolean isOdd() { return num % 2 != 0; }

    /** @return true if number is even */
    boolean isEven() { return num % 2 == 0; }
}

```

```

public static void main(String[] args) {
    int val = args.length > 0 ? Integer.parseInt(args[0]) : 0;
    MyNumber n = new MyNumber(val);
    System.out.println("Number: " + val);
    System.out.println("Negative: " + n.isNegative());
    System.out.println("Positive: " + n.isPositive());
    System.out.println("Zero: " + n.isZero());
    System.out.println("Odd: " + n.isOdd());
    System.out.println("Even: " + n.isEven());
}
}

```

Sample Output (argument -4):

```

Number: -4
Negative: true
Positive: false
Zero: false
Odd: false
Even: true

```

Slip 12

Q.1

Question:

Define an interface Operation with methods area() and volume() and constant PI=3.142.
 Create class Cylinder implementing Operation, members radius, height.
 Create object and calculate area and volume.

Program:

```

interface Operation {
    double PI = 3.142;
    double area();
    double volume();
}

class Cylinder implements Operation {
    double r, h;
    Cylinder(double r, double h) {
        this.r = r;
        this.h = h;
    }
    public double area() {
        return 2 * PI * r * (h + r);
    }
    public double volume() {
        return PI * r * r * h;
    }
    public static void main(String[] args) {
        Cylinder c = new Cylinder(3, 5);
        System.out.println("Area = " + c.area());
    }
}

```

```
        System.out.println("Volume = " + c.volume());
    }
}
```

Sample Output:

Area = 150.816

Volume = 141.39

Q.2

Question:

Write a Java program to create a Package SY with class SYMarks, and package TY with class TYMarks.

Create Student objects, add SY + TY computer subject marks, calculate grade and display.

Program:

(same structure as Slip 3 Q.2 – can reuse that program)

Slip 13

Q.1

Question:

Write a program to find the cube of given number using function interface.

Program:

```
import java.util.function.Function;
import java.util.Scanner;

class CubeUsingFunction {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number: ");
        int n = sc.nextInt();
        Function<Integer, Integer> cube = x -> x * x * x;
        System.out.println("Cube = " + cube.apply(n));
    }
}
```

Sample Output:

Enter number: 4

Cube = 64

Q.2

Question:

Define class CricketPlayer (name, no_of_innings, no_of_times_notout, totalruns, bat_avg).

Create array of players, calculate batting average, sort by average and display.

Program:

(Same as Slip 9 Q.1)

Slip 14

Q.1

Question:

Define a class Patient (patient_name, patient_age, patient_oxy_level, patient_HRCT_report).

Handle exception if oxygen < 95% and HRCT > 10, throw user-defined exception "Patient is Covid Positive(+) and Need to Hospitalized", else display info.

Program:

```
import java.util.Scanner;

class CovidException extends Exception {
    CovidException(String msg) { super(msg); }
}

class Patient {
    String name;
    int age;
    double oxy;
    int hrct;
    Patient(String n, int a, double o, int h) {
        name = n; age = a; oxy = o; hrct = h;
    }
    void check() throws CovidException {
        if (oxy < 95 && hrct > 10)
            throw new CovidException("Patient is Covid Positive(+) and Need to Hospitalized");
        System.out.println("Patient: " + name + ", Age: " + age + ", O2: " + oxy + ", HRCT: " + hrct);
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter name age oxy hrct: ");
        Patient p = new Patient(sc.next(), sc.nextInt(), sc.nextDouble(), sc.nextInt());
        try {
            p.check();
        } catch (CovidException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Sample Output (Critical Case):

```
Enter name age oxy hrct: John 45 90 12
Patient is Covid Positive(+) and Need to Hospitalized
```

Sample Output (Normal Case):

```
Enter name age oxy hrct: Alice 30 97 5
Patient: Alice, Age: 30, O2: 97.0, HRCT: 5
```

Q.2

Question:

Create an abstract class Order (id, description) and subclasses PurchaseOrder, SalesOrder (similar to Slip 1 Q.2).

Accept and display 3 objects each.

Program:

(Same as Slip 1 Q.2)

Slip 15

Q.1

Question:

Write a program to read a text file “sample.txt” and display contents in reverse order and also display contents in uppercase.

Program:

```
import java.io.*;

class FileReverse {
    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new FileReader("sample.txt"));
        StringBuilder sb = new StringBuilder();
        String line;
        while ((line = br.readLine()) != null) sb.append(line).append("\n");
        br.close();
        System.out.println("Uppercase Content:");
        System.out.println(sb.toString().toUpperCase());
        System.out.println("Reversed Content:");
        System.out.println(sb.reverse());
    }
}
```

Sample Output:

(If file contains "Hello World")

Uppercase Content:

HELLO WORLD

Reversed Content:

dlroW olleH

Q.2

Question:

Using marker interface create class Product with product_id, product_name, product_cost, product_quantity.

Display contents and object count.

Program:

(Same as Slip 2 Q.2)

Slip 16

Q.1

Question:

Write a program to read a text file “sample.txt”, count and display number of words and lines in the file.

Program:

```
import java.io.*;

class FileWordLineCount {
```

```

public static void main(String[] args) throws Exception {
    BufferedReader br = new BufferedReader(new FileReader("sample.txt"));
    String line;
    int lines = 0, words = 0;
    while ((line = br.readLine()) != null) {
        lines++;
        String[] w = line.trim().split("\\s+");
        if (!line.trim().isEmpty()) words += w.length;
    }
    br.close();
    System.out.println("Number of lines: " + lines);
    System.out.println("Number of words: " + words);
}
}

```

Sample Output (if file contains 2 lines):

Number of lines: 2

Number of words: 7

Q.2

Question:

Write a program to copy contents of one file into another.

Program:

```

import java.io.*;

class FileCopy {
    public static void main(String[] args) throws Exception {
        FileInputStream fin = new FileInputStream("source.txt");
        FileOutputStream fout = new FileOutputStream("destination.txt");
        int ch;
        while ((ch = fin.read()) != -1) fout.write(ch);
        fin.close();
        fout.close();
        System.out.println("File copied successfully.");
    }
}

```

Sample Output:

File copied successfully.

Slip 17

Q.1

Question:

Write a program to store roll number and name of students in a file and display contents.

Program:

```

import java.io.*;
import java.util.Scanner;

```

```

class StudentFile {

```

```

public static void main(String[] args) throws Exception {
    Scanner sc = new Scanner(System.in);
    FileWriter fw = new FileWriter("students.txt");
    System.out.print("Enter number of students: ");
    int n = sc.nextInt();
    for (int i = 0; i < n; i++) {
        System.out.print("Enter roll and name: ");
        fw.write(sc.nextInt() + " " + sc.next() + "\n");
    }
    fw.close();

    System.out.println("File Contents:");
    BufferedReader br = new BufferedReader(new FileReader("students.txt"));
    String line;
    while ((line = br.readLine()) != null) System.out.println(line);
    br.close();
}
}

```

Sample Output:

```

Enter number of students: 2
Enter roll and name: 1 Alice
Enter roll and name: 2 Bob
File Contents:
1 Alice
2 Bob

```

Q.2

Question:

Write a program to count number of objects created for a class using static data member and method.

Program:

```

class ObjectCounter {
    static int count = 0;
    ObjectCounter() { count++; }
    static void displayCount() {
        System.out.println("Objects created: " + count);
    }
    public static void main(String[] args) {
        new ObjectCounter();
        new ObjectCounter();
        new ObjectCounter();
        displayCount();
    }
}

```

Sample Output:

```

Objects created: 3

```

Slip 18

Q.1

Question:

Write a program to accept the details of 'n' books (title, author, price) and display details of all books having price greater than 500.

Program:

```
import java.util.Scanner;

class Book {
    String title, author;
    double price;
    Book(String t, String a, double p) {
        title = t; author = a; price = p;
    }
    void display() {
        System.out.println(title + " by " + author + " Price: " + price);
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of books: ");
        int n = sc.nextInt();
        Book[] b = new Book[n];
        for (int i = 0; i < n; i++) {
            System.out.print("Enter title author price: ");
            b[i] = new Book(sc.next(), sc.next(), sc.nextDouble());
        }
        System.out.println("Books with price > 500:");
        for (Book book : b)
            if (book.price > 500) book.display();
    }
}
```

Sample Output:

```
Enter number of books: 2
Enter title author price: Java Kunal 450
Enter title author price: Python Rahul 650
Books with price > 500:
Python by Rahul Price: 650.0
```

Q.2

Question:

Write a program to demonstrate use of constructor overloading.

Program:

```
class Student {
    int roll;
    String name;

    Student() {
```

```

    roll = 0;
    name = "Unknown";
}

Student(int r, String n) {
    roll = r;
    name = n;
}

void display() {
    System.out.println("Roll: " + roll + ", Name: " + name);
}

public static void main(String[] args) {
    Student s1 = new Student();
    Student s2 = new Student(1, "Alice");
    s1.display();
    s2.display();
}
}

```

Sample Output:

Roll: 0, Name: Unknown

Roll: 1, Name: Alice

Slip 19

Q.1

Question:

Write a program to demonstrate use of parameterized constructor and copy constructor.

Program:

```

class Person {
    String name;
    int age;

    Person(String n, int a) {
        name = n;
        age = a;
    }

    Person(Person p) {
        name = p.name;
        age = p.age;
    }

    void display() {
        System.out.println(name + " " + age);
    }
}

```

```

public static void main(String[] args) {
    Person p1 = new Person("Alice", 25);
    Person p2 = new Person(p1);
    p1.display();
    p2.display();
}
}

```

Sample Output:

Alice 25
 Alice 25

Q.2

Question:

Write a program to create a thread using Runnable interface.

Program:

```

class MyThread implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++)
            System.out.println("Child Thread: " + i);
    }
    public static void main(String[] args) {
        Thread t = new Thread(new MyThread());
        t.start();
        for (int i = 1; i <= 5; i++)
            System.out.println("Main Thread: " + i);
    }
}

```

Sample Output (order may vary):

Main Thread: 1
 Main Thread: 2
 Child Thread: 1
 Child Thread: 2
 ...

Slip 20

Q.1

Question:

Write a program to demonstrate multithreading by creating two threads.

First thread should print numbers from 1 to 10 with 1-second delay, second should print multiplication table of 5.

Program:

```

class Thread1 extends Thread {
    public void run() {
        for (int i = 1; i <= 10; i++) {
            System.out.println("Number: " + i);
            try { Thread.sleep(1000); } catch (Exception e) {}
        }
    }
}

```

```

    }
}

class Thread2 extends Thread {
    public void run() {
        for (int i = 1; i <= 10; i++)
            System.out.println("5 x " + i + " = " + (5 * i));
    }
}

```

```

class MultiThreadDemo {
    public static void main(String[] args) {
        new Thread1().start();
        new Thread2().start();
    }
}

```

Sample Output (interleaved):

```

Number: 1
5 x 1 = 5
5 x 2 = 10
Number: 2
5 x 3 = 15
...

```

Q.2

Question:

Write a program to display numbers from 1 to 10 using thread priority.

Program:

```

class PriorityThread extends Thread {
    PriorityThread(String name, int priority) {
        super(name);
        setPriority(priority);
    }
    public void run() {
        for (int i = 1; i <= 10; i++)
            System.out.println(getName() + ": " + i);
    }
    public static void main(String[] args) {
        PriorityThread t1 = new PriorityThread("Low", Thread.MIN_PRIORITY);
        PriorityThread t2 = new PriorityThread("High", Thread.MAX_PRIORITY);
        t1.start();
        t2.start();
    }
}

```

Sample Output (order depends on scheduler):

```

Low: 1
High: 1

```

High: 2

Low: 2

...

Slip 21

Q.1

Question:

Write a program to create two threads:

First thread prints numbers from 1 to 10.

Second thread prints numbers from 20 to 30.

Program:

```
class FirstThread extends Thread {
    public void run() {
        for (int i = 1; i <= 10; i++)
            System.out.println("First Thread: " + i);
    }
}

class SecondThread extends Thread {
    public void run() {
        for (int i = 20; i <= 30; i++)
            System.out.println("Second Thread: " + i);
    }
}

class TwoThreads {
    public static void main(String[] args) {
        new FirstThread().start();
        new SecondThread().start();
    }
}
```

Sample Output (interleaved):

First Thread: 1

Second Thread: 20

First Thread: 2

Second Thread: 21

...

Q.2

Question:

Write a program to create a thread which prints "I am child thread" five times.

Main thread should print "I am main thread" five times.

Program:

```
class ChildThread extends Thread {
    public void run() {
        for (int i = 0; i < 5; i++)
            System.out.println("I am child thread");
    }
}
```



```

    }
}

class MainChild {
    public static void main(String[] args) {
        new ChildThread().start();
        for (int i = 0; i < 5; i++)
            System.out.println("I am main thread");
    }
}

```

Sample Output (interleaved):

```

I am main thread
I am child thread
I am main thread
I am child thread
...

```

Slip 22

Q.1

Question:

Write a program to read and display contents of file using FileInputStream.

Program:

```

import java.io.*;

class FileRead {
    public static void main(String[] args) throws Exception {
        FileInputStream fin = new FileInputStream("sample.txt");
        int ch;
        while ((ch = fin.read()) != -1)
            System.out.print((char) ch);
        fin.close();
    }
}

```

Sample Output (if file contains “Hello”):

```

Hello

```

Q.2

Question:

Write a program to create a file and write data using FileOutputStream.

Program:

```

import java.io.*;

class FileWrite {
    public static void main(String[] args) throws Exception {
        FileOutputStream fout = new FileOutputStream("output.txt");
        String data = "This is sample data written to file.";
        fout.write(data.getBytes());
    }
}

```

```
fout.close();
System.out.println("Data written successfully.");
}
}
```

Sample Output:

Data written successfully.

Slip 23**Q.1****Question:**

Write a program to perform file operations using RandomAccessFile:

Write, read, seek operations.

Program:

```
import java.io.*;

class RandomFileDemo {
    public static void main(String[] args) throws Exception {
        RandomAccessFile raf = new RandomAccessFile("random.txt", "rw");
        raf.writeBytes("Hello Random Access File");
        raf.seek(0);
        String line = raf.readLine();
        System.out.println("Read: " + line);
        raf.seek(6);
        System.out.println("From position 6: " + raf.readLine());
        raf.close();
    }
}
```

Sample Output:

Read: Hello Random Access File

From position 6: Random Access File

Q.2**Question:**

Write a program to implement synchronized thread for printing table of given number.

Program:

```
class Table {
    synchronized void printTable(int n) {
        for (int i = 1; i <= 10; i++) {
            System.out.println(n + " x " + i + " = " + (n * i));
            try { Thread.sleep(500); } catch (Exception e) {}
        }
    }
}

class MyThread1 extends Thread {
    Table t; MyThread1(Table t) { this.t = t; }
    public void run() { t.printTable(5); }
```

```
}
```

```
class MyThread2 extends Thread {  
    Table t; MyThread2(Table t) { this.t = t; }  
    public void run() { t.printTable(7); }  
}
```

```
class SyncTable {  
    public static void main(String[] args) {  
        Table obj = new Table();  
        new MyThread1(obj).start();  
        new MyThread2(obj).start();  
    }  
}
```

Sample Output (sequential because of synchronization):

5 x 1 = 5

...

5 x 10 = 50

7 x 1 = 7

...

7 x 10 = 70

Slip 24

Q.1

Question:

Write a program to implement inter-thread communication using wait() and notify().

Program:

```
class SharedData {  
    int value;  
    boolean available = false;  
  
    synchronized void put(int v) {  
        while (available)  
            try { wait(); } catch (Exception e) {}  
        value = v;  
        available = true;  
        System.out.println("Put: " + v);  
        notify();  
    }  
  
    synchronized int get() {  
        while (!available)  
            try { wait(); } catch (Exception e) {}  
        available = false;  
        System.out.println("Got: " + value);  
        notify();  
        return value;  
    }  
}
```

```

    }
}

class Producer extends Thread {
    SharedData s;
    Producer(SharedData s) { this.s = s; }
    public void run() {
        for (int i = 1; i <= 5; i++) s.put(i);
    }
}

```

```

class Consumer extends Thread {
    SharedData s;
    Consumer(SharedData s) { this.s = s; }
    public void run() {
        for (int i = 1; i <= 5; i++) s.get();
    }
}

```

```

class InterThreadDemo {
    public static void main(String[] args) {
        SharedData s = new SharedData();
        new Producer(s).start();
        new Consumer(s).start();
    }
}

```

Sample Output (order alternates):

```

Put: 1
Got: 1
Put: 2
Got: 2
...

```

Q.2

Question:

Write a program to demonstrate use of join() method.

Program:

```

class JoinDemo extends Thread {
    public void run() {
        for (int i = 1; i <= 3; i++)
            System.out.println(getName() + " " + i);
    }
    public static void main(String[] args) throws Exception {
        JoinDemo t1 = new JoinDemo();
        JoinDemo t2 = new JoinDemo();
        t1.start();
        t1.join();
    }
}

```

```
        t2.start();
    }
}
```

Sample Output:

```
Thread-0 1
Thread-0 2
Thread-0 3
Thread-1 1
Thread-1 2
Thread-1 3
```

Slip 25**Q.1****Question:**

Write a program to demonstrate use of yield() method.

Program:

```
class YieldDemo extends Thread {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(getName() + " " + i);
            Thread.yield();
        }
    }
}

public static void main(String[] args) {
    YieldDemo t1 = new YieldDemo();
    YieldDemo t2 = new YieldDemo();
    t1.start();
    t2.start();
}
```

Sample Output (interleaved):

```
Thread-0 1
Thread-1 1
Thread-0 2
Thread-1 2
...
```

Q.2**Question:**

Write a program to create a simple calculator using AWT with buttons for +, -, *, / and textfields to display result.

Program:

```
import java.awt.*;
import java.awt.event.*;

class Calculator extends Frame implements ActionListener {
    TextField t1, t2, t3;
```

Button add, sub, mul, div;

```
Calculator() {  
    setLayout(new GridLayout(4, 2));  
    t1 = new TextField(); t2 = new TextField(); t3 = new TextField();  
    add(new Label("Num1:")); add(t1);  
    add(new Label("Num2:")); add(t2);  
    add(new Label("Result:")); add(t3);  
    Panel p = new Panel();  
    add = new Button("+"); sub = new Button("-"); mul = new Button("*"); div = new Button("/");  
    p.add(add); p.add(sub); p.add(mul); p.add(div);  
    add(p);  
    add.addActionListener(this); sub.addActionListener(this);  
    mul.addActionListener(this); div.addActionListener(this);  
    setSize(300,200); setVisible(true);  
}
```

```
public void actionPerformed(ActionEvent e) {  
    double a = Double.parseDouble(t1.getText());  
    double b = Double.parseDouble(t2.getText());  
    double res = 0;  
    if (e.getSource() == add) res = a + b;  
    if (e.getSource() == sub) res = a - b;  
    if (e.getSource() == mul) res = a * b;  
    if (e.getSource() == div) res = a / b;  
    t3.setText("" + res);  
}
```

```
public static void main(String[] args) {  
    new Calculator();  
}
```

```
}
```

(Output is GUI-based; user enters numbers and clicks buttons)

Slip 26

Q.1

Question:

Write a program to demonstrate use of `isAlive()` and `join()` methods.

Program:

```
class AliveJoinDemo extends Thread {  
    public void run() {  
        for (int i = 1; i <= 3; i++)  
            System.out.println(getName() + " " + i);  
    }  
    public static void main(String[] args) throws Exception {  
        AliveJoinDemo t1 = new AliveJoinDemo();  
        AliveJoinDemo t2 = new AliveJoinDemo();  
    }  
}
```

```

        System.out.println("Before start t1 alive: " + t1.isAlive());
        t1.start();
        System.out.println("After start t1 alive: " + t1.isAlive());
        t1.join();
        t2.start();
        System.out.println("t2 alive after start: " + t2.isAlive());
    }
}

```

Sample Output:

```

Before start t1 alive: false
After start t1 alive: true
Thread-0 1
Thread-0 2
Thread-0 3
t2 alive after start: true
Thread-1 1
Thread-1 2
Thread-1 3

```

Q.2

Question:

Write a program to demonstrate use of `setPriority()` and `getPriority()` methods.

Program:

```

class PriorityDemo extends Thread {
    public void run() {
        System.out.println(getName() + " Priority: " + getPriority());
    }
    public static void main(String[] args) {
        PriorityDemo t1 = new PriorityDemo();
        PriorityDemo t2 = new PriorityDemo();
        t1.setPriority(Thread.MIN_PRIORITY);
        t2.setPriority(Thread.MAX_PRIORITY);
        t1.start();
        t2.start();
    }
}

```

Sample Output:

```

Thread-0 Priority: 1
Thread-1 Priority: 10

```

Slip 27

Q.1

Question:

Write a program to demonstrate `Runnable` interface with multiple threads performing different tasks.

Program:

```

class Task1 implements Runnable {

```

```

public void run() {
    for (int i = 1; i <= 5; i++)
        System.out.println("Task1 counting: " + i);
}
}

```

```

class Task2 implements Runnable {
    public void run() {
        for (char c = 'A'; c <= 'E'; c++)
            System.out.println("Task2 printing: " + c);
    }
}

```

```

class RunnableDemo {
    public static void main(String[] args) {
        new Thread(new Task1()).start();
        new Thread(new Task2()).start();
    }
}

```

Sample Output (interleaved):

```

Task1 counting: 1
Task2 printing: A
Task1 counting: 2
Task2 printing: B
...

```

Q.2

Question:

Write a program to implement a simple GUI screen using Swing to accept a user name and display welcome message.

Program:

```

import javax.swing.*;
import java.awt.event.*;

class WelcomeGUI extends JFrame implements ActionListener {
    JTextField nameField;
    JButton submit;
    JLabel result;

    WelcomeGUI() {
        setTitle("Welcome App");
        setLayout(null);
        JLabel l = new JLabel("Enter Name:");
        l.setBounds(20, 20, 100, 30);
        add(l);

        nameField = new JTextField();

```



```

nameField.setBounds(120, 20, 150, 30);
add(nameField);

submit = new JButton("Submit");
submit.setBounds(100, 70, 100, 30);
add(submit);

result = new JLabel();
result.setBounds(20, 120, 250, 30);
add(result);

submit.addActionListener(this);
setSize(300, 200);
setVisible(true);
setDefaultCloseOperation(EXIT_ON_CLOSE);
}

public void actionPerformed(ActionEvent e) {
    result.setText("Welcome " + nameField.getText() + "!");
}

public static void main(String[] args) {
    new WelcomeGUI();
}
}

```

(GUI Output: Displays a window, user types name, clicks Submit → “Welcome <name>!” appears)

Slip 28

Q.1

Question:

Write a program to implement multiple catch blocks for handling different exceptions.

Program:

```

import java.util.Scanner;

class MultipleCatch {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter number: ");
            int n = sc.nextInt();
            System.out.print("Enter divisor: ");
            int d = sc.nextInt();
            int result = n / d;
            System.out.println("Result: " + result);
            int[] arr = new int[3];
            System.out.println(arr[5]); // Will cause exception
        }
    }
}

```

```

    } catch (ArithmeticException e) {
        System.out.println("Cannot divide by zero.");
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Array index out of range.");
    } catch (Exception e) {
        System.out.println("Some other exception: " + e);
    }
}
}

```

Sample Output (if divisor=0):

Enter number: 10
Enter divisor: 0
Cannot divide by zero.

Sample Output (valid divisor):

Enter number: 10
Enter divisor: 2
Result: 5
Array index out of range.

Q.2

Question:

Write a program to demonstrate use of throw and throws keywords.

Program:

```

class AgeException extends Exception {
    AgeException(String msg) { super(msg); }
}

class ThrowThrowsDemo {
    static void checkAge(int age) throws AgeException {
        if (age < 18) throw new AgeException("Age must be >= 18");
        System.out.println("Eligible to vote.");
    }

    public static void main(String[] args) {
        try {
            checkAge(15);
        } catch (AgeException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}

```

Sample Output:

Exception: Age must be >= 18